Episodio 14 de 14: Ensayo para la exposición: las preguntas del jurado

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 13, «Juicio a la réplica (2): corrección, RAM, estadística y sentencia».
- **Episodio siguiente:** es el último de la serie.

Lo esencial de este episodio:

- La réplica contada en 20 segundos, en 1 minuto y en 3 minutos.
- Las 20 preguntas más probables del jurado, cada una con su respuesta corta.
- Las palabras que conviene evitar, qué decir en su lugar y las cifras clave para recordar.

---

1. La réplica en tres tamaños

Aprende las tres versiones. La corta es para cuando te interrumpen; la larga, para cuando te dejan hablar.

En 20 segundos:

> «Repetimos el paper con los programas oficiales de los autores en un PC de 24 núcleos, con y sin Docker: más de 14 000 mediciones y ninguna fallida. De las 15 afirmaciones del paper, 9 se reproducen, 2 en parte, 3 no y 1 no se confirma. Lo central sale bien: el algoritmo es correcto, cumple su cota *wait-free* y lee rápido. Lo que no sale es que gane a Folly.»

En 1 minuto, añade el *cómo*:

> «Usamos el artefacto publicado de los autores, con su suma MD5 comprobada, así que medimos su código y no una copia nuestra. El PC no tiene 4 procesadores como la máquina del paper, así que dividimos los hilos entre 6 y simulamos la distancia entre procesadores colocando cada hilo en núcleos concretos. Recortamos las mediciones de 10 a 2 segundos para que cupieran en una noche, y luego comprobamos con la receta exacta cuánto movía eso los números. También medimos si Docker influía: no. Y pasamos detectores de errores de memoria a todos los rivales, no solo al algoritmo del paper.»

En 3 minutos, cuenta la historia completa en este orden:

1. **El problema:** leer un puntero compartido y sumar 1 a su contador no es atómico. Entre las dos cosas, otro hilo puede liberar el objeto. Es la carrera *read-reclaim*.
2. **La idea del paper:** la lectura usa un solo `fetch_add`, que nunca falla, y por eso es *wait-free*.
3. **Lo que se reproduce:** gana a Libstdc++ y a Williams; Anderson gana con muchas escrituras, como dice el paper; no retiene memoria; la cota de reintentos se cumple con mucho margen.

4. **Lo que no se reproduce, y por qué no es un error nuestro:** Folly ya rinde distinto con **un solo hilo**, donde no hay pelea entre hilos; y con un millón de pilas todos chocan con el techo de la RAM de este PC.
5. **Lo que descubrimos por el camino:** el programa del equipo tenía compartición falsa, el binario oficial de latencia de Williams llevaba un candado escondido, y alinear las pilas frena con un millón de pilas porque dejan de caber en la caché.
6. **Los límites:** sin 4 sockets, sin NUMA, la latencia con contención es ruidosa y la RAM es de un solo módulo.

---

6. Ensayo: 20 preguntas del jurado

Respuestas cortas. Si piden más, la sección indicada tiene el detalle.

1. **¿Por qué no una máquina virtual con 4 nodos NUMA?** Porque sería un decorado: el sistema creería que hay 4 nodos, pero ningún viaje tardaría más. Preferimos distancias reales y medidas. (Parte 5.1)
2. **¿Por qué dividisteis los hilos entre 6?** Para conservar la proporción: 144 hilos eran toda la máquina del paper y 24 son toda esta. (Parte 4.3)
3. **¿Por qué 2 s y no 10?** Para que la figura 5 cupiera en una hora en vez de cinco. Después medimos cuánto cambiaba: el orden se mantiene en 4 de 6 escenarios. (Parte 6.7)
4. **¿Por qué orden aleatorio y un proceso por punto?** Para que el calor o una interrupción afecten a todos por igual, y que ningún punto herede el estado del anterior. (Parte 4.3)
5. **¿Usasteis el código de los autores?** Sí: el artefacto de Zenodo (`10.5281/zenodo.11775182`), con el MD5 comprobado. `bench.cpp` del equipo es una segunda opinión. (Parte 4.2)
6. **¿Qué significa *wait-free* y cómo lo comprobasteis?** Que cada operación termina en un número acotado de pasos, haga lo que haga el resto. Contamos los intentos en una copia del código: 8 como mucho, con una cota de 32. (Parte 6.5)
7. **¿Por qué Folly sale tan distinto?** No lo sabemos. Sí sabemos que no es contención, porque la diferencia ya existe con un hilo, y que no es nuestro montaje, porque Williams sí coincide. (Parte 6.1)
8. **¿Por qué empatan todos con un millón de pilas?** Muy probablemente por la RAM: tres algoritmos llegan al 89-93 % del techo de este PC, que tiene un solo módulo. (Parte 6.1)
9. **¿Qué aporta vuestra réplica que no esté en el paper?** Williams en latencia, que el paper omitió; el candado escondido por compilar sin `-mcx16`; la batería de afinidad; y la medición real de la RAM. (Partes 6.2, 6.4 y 6.1)
10. **¿Influye Docker?** No: de 270 puntos, 259 cambian menos de un 10 %. (Parte 6.6)
11. **¿Cómo sabéis que ASan funcionaba?** Porque la versión ingenua lo hace saltar enseguida, y comprobamos que los binarios estaban instrumentados. (Parte 6.5)
12. **¿Qué es la compartición falsa?** Dos personas que escriben en renglones distintos de la misma hoja y tienen que pasarse el cuaderno cada vez. Datos distintos, misma línea de caché. (Parte 6.6)
13. **¿Por qué alinear frena con un millón de pilas?** Porque cada pila pasa de 8-16 bytes a 128, y el millón deja de caber en la caché L3. Medido: cada operación saca de la RAM entre 207 y 265 bytes más. (Parte 6.6)

14. **¿Por qué Williams se hunde entre clústeres?** No lo sabemos. Descartamos un par de núcleos defectuoso, Docker y `boost::atomic`. Solo pasa con el programa oficial. (Parte 6.4)
15. **¿Qué pasa con más hilos que núcleos?** Se mide (30 y 36 hilos, como 180 y 216 en el paper), pero hay que leerlo con cuidado. En latencia, p = 56 se declara no comparable. (Partes 4.3 y 6.1)
16. **¿Por qué Anderson gana cuando hay escrituras?** Porque en Autores cada lectura modifica el puntero compartido, y eso hace fallar los `CAS` de quien quiere escribir. En Anderson, leer no lo toca. (Parte 6.1)
17. **¿Se puede repetir?** Sí: cada experimento es una orden y se reanuda solo. Están en el README de `replica/completa/`, con lo que tarda cada uno. (Parte 9)
18. **¿Cuánta memoria retiene Anderson?** Hasta 19 960 bloques (469 KB) con 24 hilos. Autores, nada por encima del límite. (Parte 6.3)
19. **Con una semana más, ¿qué haríais?** Un segundo módulo de RAM, intervalos de confianza y la figura 5 con la receta exacta. (Sección 5)
20. **Vuestra conclusión, en una frase.** «La idea central del paper sale bien parada: es correcto, cumple su cota, no acumula basura y lee rápido. No se sostiene la superioridad frente a Folly ni la ventaja con muchísimas pilas, y la escala de 4 sockets no se pudo reproducir.» (Parte 7)

---

7. Lo que no hay que decir, y qué decir en su lugar

Muchas críticas nacen de una palabra demasiado fuerte. Cambia la palabra y la crítica desaparece.

No digas	Di	Por qué
«Demostramos que es correcto»	«No encontramos errores buscándolos, y la cota se cumplió con margen»	Una prueba nunca demuestra corrección (cargo 8)
«Simulamos 4 Xeon»	«Simulamos la distancia entre sockets, de forma más suave; NUMA no»	1,35× frente a unas 3× (cargo 2)
«El paper se equivoca con Folly»	«En nuestra máquina Folly rinde mucho mejor, ya con un hilo; la causa está abierta»	No tenemos su máquina (cargo 4)
«Docker no influye»	«Lo medimos: 259 de 270 puntos cambian menos de un 10 %»	La cifra convence más que el adjetivo (cargo 6)
«La RAM es el techo»	«Tres algoritmos llegan al 89-93 % del techo de la RAM; lo refuerza, no lo demuestra»	Falta el segundo módulo (cargo 9)
«Autores gana» por un 5 %	«Empatan»	Las diferencias pequeñas están dentro del ruido (cargos 3 y 11)
«Es una réplica exacta»	«Es una reproducción con los programas oficiales, a otra escala»	Terminología (sección 2)

No digas	Di	Por qué
«Eso es culpa de nuestro PC»	«Eso depende de la máquina, y lo marcamos como “en este PC”»	Suena a excusa; lo otro, a método

---

8. Chuleta de cifras

Qué	Cifra
Mediciones	Más de 14 000, ninguna fallida
Veredicto de las 15 afirmaciones	9 se reproducen, 2 en parte, 3 no y 1 no se confirma
Máquina	Core Ultra 9 285: 24 núcleos (8 P + 16 E), sin SMT, 1 nodo NUMA, un módulo de 32 GB DDR5-5600
Máquina del paper (figura 5)	4 Xeon E7-8867: 72 núcleos, 144 hilos
Hilos	Los del paper divididos entre 6: 24 aquí equivale a 144 allí
Cota *wait-free*	Máximo 8 intentos con P = 32, en 8 061 142 recargas
Romper la regla C ≥ 2P	42 348 errores
Detectores en los rivales	26 pruebas, 0 avisos
Mejor caso de Autores (n = 10, u = 1 %, 24 hilos)	Autores 122, Anderson 74, Folly 46, Williams 35 y Libstdc++ 11 Mop/s
Anderson con muchas escrituras (n = 1)	1,87× a 1,99× de Autores
Folly con un hilo	0,20× en el paper frente a 0,86× aquí (n = 1, u = 1 %)
Docker frente a sin Docker	259 de 270 puntos cambian menos de un 10 %
Recortes, en los 6 escenarios	De −10 % a +19 %; el orden se mantiene en 4 de 6
Latencias entre núcleos	35 ns en el mismo clúster, 47 entre clústeres, 86 entre núcleos P
Separar 4 hilos	Autores pierde un 32-54 % con contención; con n = 10^6, ±2 %
Techo de la RAM	43 GB/s en orden, 37 al azar
Memoria retenida por Anderson	Hasta 19 960 bloques, 469 KB, con 24 hilos
Compartición falsa en `bench.cpp`	Autores, de 35 a 120 Mop/s al alinear (n = 10)
Alinear con n = 10^6	−24 a −27 %, con 207-265 bytes más de RAM por operación
Williams entre clústeres	−97 % con Docker, −76 % sin Docker
